



INSTITUTO PROFESIONAL
SAN SEBASTIAN

EXAMEN FINAL TRANSVERSAL

TALLER DE DATOS I: MODELAMIENTO (CD201ICDA) MODELO
HÍBRIDO DE DATOS PARA LA GESTIÓN DE SOLICITUDES
CIUDADANAS EN UN MUNICIPIO CHILENO

Estudiante: Cristian Castañeda Anhy Gutierrez

Asignatura: Taller de Datos I Modelamiento

Profesor: Ivan Bilbao

Fecha: 12 de Junio de 2026

ACREDITACIÓN 2025
*Construimos juntos una
formación de calidad*





Introducción

El presente informe documenta el diseño e implementación de un modelo de datos híbrido para el sistema de gestión de solicitudes ciudadanas de una municipalidad chilena. El caso requiere integrar datos de naturaleza heterogénea, recibidos a través de tres canales distintos (sitio web, aplicación móvil y oficina presencial), y satisfacer simultáneamente dos clases de necesidades: la gestión operativa en tiempo real del ciclo de vida de cada solicitud, y el análisis estratégico de patrones territoriales, tipológicos y de reincidencia que permita al municipio tomar decisiones informadas.

La solución propuesta combina un núcleo relacional normalizado sobre PostgreSQL 15+ con campos JSONB validados mediante **JSON Schema**, lo que permite mantener la integridad transaccional propia de un modelo ACID mientras se conserva la flexibilidad necesaria para representar los atributos variables que cada tipo de solicitud trae consigo. Esta arquitectura híbrida responde directamente a la triple naturaleza de los datos que describe el enunciado: estructurados (fechas, tipos, estados), semiestructurados (JSON con geolocalización y metadatos del canal) y no estructurados (descripción libre del ciudadano e imágenes adjuntas).

Alcance del trabajo

El proyecto entrega un modelo de datos completo, ejecutable y validado, acompañado de los siguientes artefactos:

- Script SQL con el esquema relacional normalizado (PK, FK, CHECK, UNIQUE, NOT NULL) y la carga de catálogos base.
- Script SQL con la batería de pruebas de integridad: 1 prueba positiva y 20 pruebas negativas que cubren los cuatro criterios de calidad de la norma ISO/IEC 25012.
- Dataset sintético de 400 solicitudes con patrones realistas embebidos (reincidencia, concentración sectorial, retraso de SLA).





- Esquema JSON Schema (Draft 2020-12) con discriminador if/then/else para validar el campo metadatos.
- Validador Python que ejecuta el JSON Schema sobre los 400 registros sintéticos y sobre 12 casos inválidos diseñados a propósito.
- Notebook Jupyter integrador que orquesta el flujo completo, ejecuta las consultas operativas y analíticas, y genera visualizaciones.
- Diccionario de Datos formal con 83 columnas y 23 campos JSONB documentados.

Análisis de Necesidades

El levantamiento de necesidades se desarrolló a partir del enunciado del caso, clasificándolas en tres dimensiones según su horizonte temporal y la audiencia que las consume.

Clasificación de necesidades

Tipo	Necesidad identificada	Stakeholder
Operativa	Registrar solicitudes desde los tres canales con trazabilidad única vía folio.	Funcionario de atención
Operativa	Asignar cada solicitud a la unidad municipal responsable según el tipo.	Coordinador operativo
Operativa	Actualizar el estado a lo largo del ciclo de vida con auditoría completa.	Cuadrilla en terreno
Operativa	Adjuntar evidencia (imágenes, descripción libre, geolocalización).	Ciudadano / Funcionario
Analítica	Calcular tasa de solicitudes por sector y por tipo.	Jefatura operativa
Analítica	Medir tiempo promedio de respuesta por unidad y tipo.	Control de gestión
Analítica	Detectar ciudadanos con alta reincidencia.	Atención a usuarios
Analítica	Comparar volumen entre canales de ingreso.	Transformación digital
Estratégica	Identificar zonas críticas recurrentes (mapas de calor).	Alcaldía / SECPLA
Estratégica	Detectar patrones temporales y estacionalidad.	Planificación territorial





Estratégica	Priorizar inversión en sectores más demandantes.	Concejo municipal
Estratégica	Evaluar eficacia comparada entre unidades.	Dirección de control

Tabla 1. Clasificación de necesidades del cliente por horizonte temporal.

Volumen estimado

Para una municipalidad mediana chilena (200.000–400.000 habitantes) se estima un volumen de entre 8.000 y 15.000 solicitudes mensuales, equivalente a aproximadamente 150.000 solicitudes anuales con un crecimiento proyectado del 20% por la migración creciente desde canales presenciales hacia digitales. Esta magnitud no exige arquitecturas distribuidas de tipo big data; PostgreSQL con índices adecuados es suficiente.

Restricciones Técnicas y Normativas

Restricciones técnicas

- **Infraestructura municipal acotada:** se asume un servidor on-premise o cloud básico, sin arquitectura distribuida.
- **Motor de base de datos:** PostgreSQL 15+, justificado por ser open source, soportar el tipo JSONB nativo (clave para el modelo híbrido) y contar con extensiones geoespaciales (PostGIS) si se requiere escalamiento futuro.
- **Conectividad heterogénea:** la app móvil opera con conectividad variable en terreno, lo que exige un diseño tolerante a sincronización asincrónica.
- **Almacenamiento de binarios:** las imágenes adjuntas se externalizan a un almacén de objetos (S3, MinIO o disco local) y la base de datos solo referencia su URI.

Restricciones normativas

- **Ley N° 21.719 sobre protección de datos personales:** exige principio de minimización, finalidad declarada, derechos ARCO+P y registro de





operaciones de tratamiento. Se materializa en el modelo mediante los campos `consentimiento_datos` y `fecha_consentimiento` en la tabla `ciudadano`.

- **Ley N° 19.880 de procedimientos administrativos:** define plazos máximos de respuesta para los actos administrativos. Se materializa en la columna `sla_horas` de tipo `solicitud` y se monitorea con `fecha_limite_respuesta` en `solicitud`.
- **Ley N° 20.285 de transparencia activa:** exige que la información agregada pueda publicarse de forma anonimizada. El modelo permite generar estadísticas a nivel de sector y tipo sin exponer identidades.

Restricciones de calidad de datos

- **Incompletitud:** las solicitudes presenciales suelen ingresar con campos faltantes. Se mitiga con NOT NULL en campos críticos y con la regla CHECK que exige al menos un canal de contacto en cada ciudadano.
- **Duplicidad:** una misma incidencia puede ser reportada por múltiples ciudadanos. Se acepta a nivel de modelo porque cada reporte constituye una solicitud legítima; la consolidación es responsabilidad de procesos analíticos posteriores.
- **Inconsistencia semántica:** el texto libre puede generar etiquetas dispares. Se mitiga con catálogos enumerados y CHECK constraints sobre dominios.
- **Sesgo de canal:** la app móvil subrepresenta adultos mayores y zonas con baja penetración digital. No se mitiga a nivel de modelo pero se reconoce explícitamente en el análisis.

Formulación del Problema de Modelamiento

Sobre la base del análisis precedente, el problema se formula en los siguientes términos:





Diseñar e implementar un modelo de datos híbrido para un sistema municipal de gestión de solicitudes ciudadanas, que integre un núcleo relacional normalizado (PostgreSQL) para el flujo transaccional y la integridad referencial, con estructuras semiestructuradas (JSON validadas por JSON Schema) para datos heterogéneos como geolocalización, metadatos del canal de ingreso y atributos variables por tipo de solicitud. El modelo debe garantizar trazabilidad, soporte a consultas operativas en tiempo real y agregaciones analíticas para la toma de decisiones estratégicas, respetando las restricciones de la Ley 21.719 sobre protección de datos personales.

Criterios de calidad verificables

Aplicando el marco ISO/IEC 25012, se definen cuatro dimensiones operacionalizadas como criterios verificables:

Dimensión	Mecanismo en el modelo	Métrica
Claridad	Nombres snake_case autoexplicativos, COMMENT ON en SQL, Diccionario de Datos formal.	100% de columnas críticas documentadas.
Precisión	CHECK para dominios cerrados, regex de RUT y email, coherencia temporal, JSON Schema.	0 inserts inválidos aceptados.
Complejidad	NOT NULL en columnas críticas, CHECK length, CHECK semántico de contacto.	0% de NULL en campos críticos.
Consistencia	FK en todas las relaciones, CASCADE solo donde corresponde, UNIQUE, trigger de sincronización.	0 registros huérfanos en FK.

Tabla 2. Criterios de calidad operacionalizados según ISO/IEC 25012.

Descripción del Proceso de Modelamiento Aplicado

Identificación

Se relevaron los tres canales de ingreso, los actores involucrados (ciudadano, funcionario, cuadrilla) y los datos heterogéneos presentes en el caso. Se





identificaron también los catálogos naturales del dominio: tipos de solicitud, estados del ciclo de vida, unidades municipales responsables y sectores geográficos.

Preparación

Se realizó la limpieza conceptual del dominio, separando los datos genuinamente tabulares de los que varían por tipo de solicitud. Esta etapa fundamenta la decisión del modelo híbrido: los catálogos son tabulares puros; en cambio, los atributos específicos de cada tipo (un bache tiene profundidad y ancho, una luminaria tiene número de poste y tipo de falla) varían lo suficiente como para que un esquema relacional plano sufriera una explosión de columnas opcionales mayormente nulas.

Estructuración

Se diseñó el Modelo Entidad-Relación con doce entidades agrupadas por rol funcional, se normalizó hasta la Tercera Forma Normal y se definieron las restricciones de integridad: claves primarias, foráneas, CHECK y UNIQUE. En paralelo se diseñó la estructura del campo metadatos JSONB y su esquema de validación JSON Schema, con un discriminador (atributos.tipo) que activa el patrón if/then/else según el tipo de solicitud.

Validación

Se diseñó una batería de 20 pruebas negativas que vulneran intencionalmente cada restricción del modelo relacional, más 12 casos inválidos del JSON Schema, más 5 patrones inyectados en los datos sintéticos. La ejecución completa produce 437 evaluaciones automatizadas reproducibles, documentadas en Evidencias_Validacion.pdf.

Ajuste

Se incorporaron índices de soporte: B-tree sobre las claves foráneas frecuentes y un índice GIN sobre el campo JSONB. También se introdujo una denormalización controlada del estado actual (campo estado_actual_id en solicitud) para evitar JOIN





al historial en cada consulta operativa, manteniendo la sincronización mediante trigger.

Comparación y Justificación Técnica

Matriz comparativa de técnicas

Criterio	Modelo Relacional (PostgreSQL)	Modelo Documental (JSON)
Consistencia	Alta (ACID, transacciones, rollback)	Eventual
Flexibilidad del esquema	Baja: cambios requieren migraciones	Alta: cada documento puede variar
Integridad referencial	Fuerte: FK con CASCADE, RESTRICT	Débil: referencias por convención
Escalabilidad horizontal	Limitada: requiere sharding manual	Alta: nativamente distribuida
Validación de dominio	Nativa (CHECK, enumerados)	Externa (JSON Schema en aplicación)
Consultas analíticas	Excelente con SQL + window functions	Requiere agregaciones específicas

Tabla 3. Comparación entre el modelo relacional y el documental.

¿Por qué usar el modelo relacional para el flujo operativo?

El flujo operativo —ingreso, asignación, ejecución, cierre— exige consistencia ACID. Un funcionario que cambia el estado de una solicitud debe ver inmediatamente el efecto de su acción. Las relaciones entre solicitud, ciudadano, unidad y funcionario son estables y bien tipadas, y se benefician de la integridad referencial fuerte. Las consultas analíticas tipo "tasa de solicitudes por sector y tipo" se expresan de forma natural y eficiente en SQL.

¿Por qué usar JSON para descripción y metadatos?

Los atributos específicos de cada tipo de solicitud varían dramáticamente. Modelar esto con columnas nullable generaría una tabla con docenas de columnas mayormente nulas; modelarlo con tablas por tipo generaría una explosión de JOINS.





El campo JSONB resuelve ambos problemas: un solo campo absorbe la variabilidad, indexable mediante GIN y validable mediante JSON Schema sin perder estructura.

¿Por qué no usar solo uno?

Optar solo por el modelo relacional implicaría un esquema rígido de evolución costosa. Optar solo por el documental implicaría perder la integridad referencial que el dominio exige —no es aceptable que una solicitud apunte a un ciudadano inexistente— y complicaría el cumplimiento de la Ley 21.719. El modelo híbrido toma lo mejor de cada paradigma: integridad ACID donde importa, flexibilidad documental donde se necesita.

Modelamiento Estructurado

Diagrama Entidad–Relación

El modelo está compuesto por doce entidades organizadas en cuatro grupos funcionales: catálogos (canal, tipo_solicitud, estado_solicitud, unidad_municipal, sector); recursos humanos (funcionario, cuadrilla); negocio (ciudadano, solicitud); y trazabilidad (historial_estado, asignacion_cuadrilla, adjunto). Las cinco entidades mínimas exigidas por el enunciado están presentes, complementadas con siete adicionales que el caso justifica.



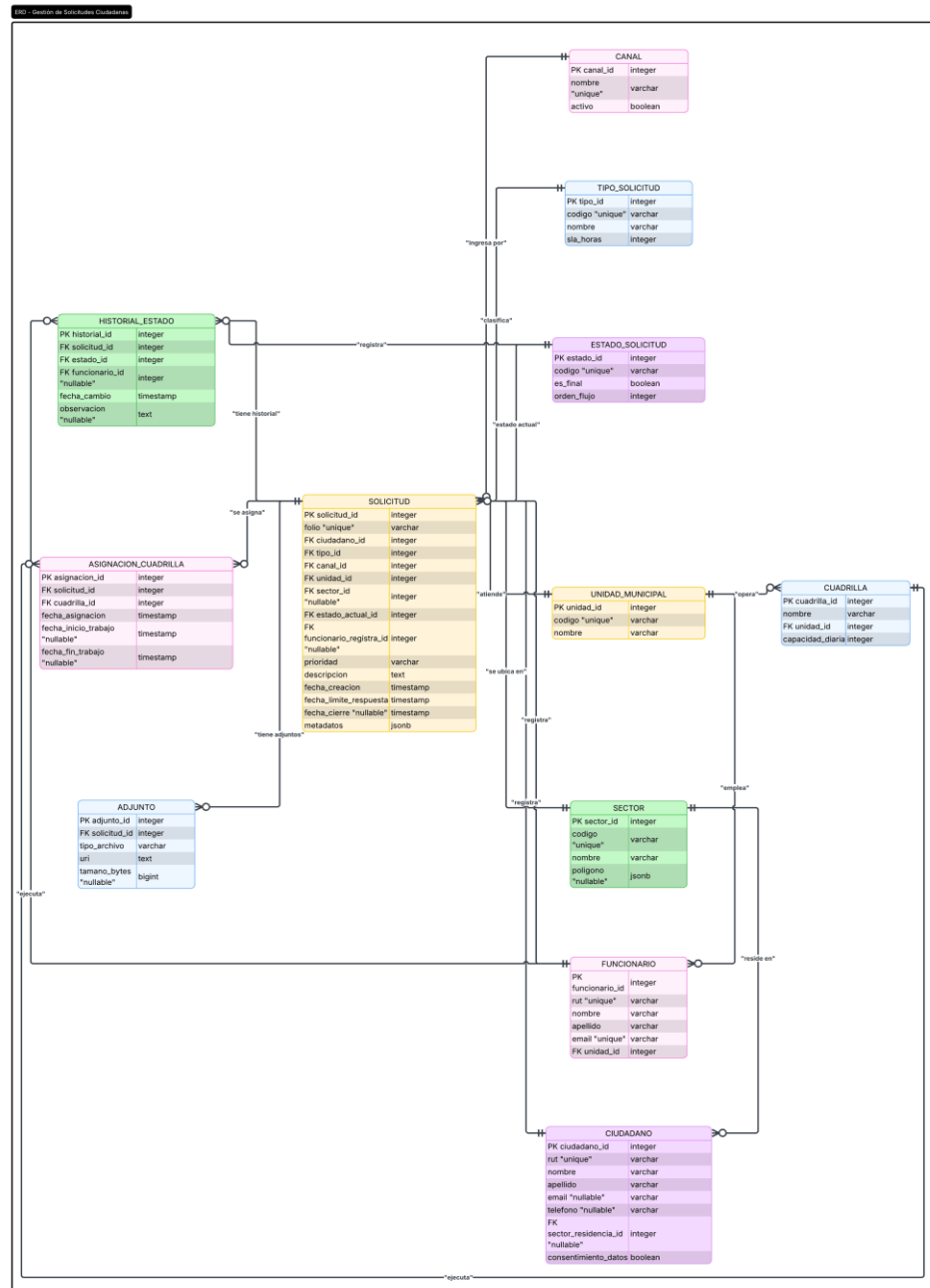


Figura 1. Diagrama Entidad-Relación del modelo (notación crow's foot).

Modelo relacional normalizado

El modelo se implementó hasta la Tercera Forma Normal con una excepción de denormalización controlada: el campo estado_actual_id en solicitud es derivable del último registro de historial_estado, pero mantenerlo denormalizado permite





responder consultas operativas sin JOIN con agregación en cada ejecución. La sincronización se mantiene mediante trigger AFTER INSERT sobre historial_estado.

A modo ilustrativo, se reproduce la definición de la tabla central solicitud (el DDL completo está en Script_Modelo.sql):

```
CREATE TABLE solicitud (
  solicitud_id          SERIAL PRIMARY KEY,
  folio                 VARCHAR(20) NOT NULL UNIQUE,
  ciudadano_id         INT NOT NULL REFERENCES ciudadano,
  tipo_id              INT NOT NULL REFERENCES tipo_solicitud,
  canal_id             INT NOT NULL REFERENCES canal,
  unidad_id            INT NOT NULL REFERENCES unidad_municipal,
  sector_id            INT REFERENCES sector,
  estado_actual_id     INT NOT NULL REFERENCES estado_solicitud,
  prioridad             VARCHAR(10) NOT NULL DEFAULT 'MEDIA'
    CHECK (prioridad IN ('BAJA', 'MEDIA', 'ALTA', 'URGENTE')),
  descripcion           TEXT NOT NULL
    CHECK (length(trim(descripcion)) >= 10),
  fecha_creacion        TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  fecha_limite_respuesta TIMESTAMP NOT NULL,
  fecha_cierre          TIMESTAMP,
  metadatos             JSONB NOT NULL DEFAULT '{} '::jsonb,
  CHECK (fecha_limite_respuesta > fecha_creacion),
  CHECK (fecha_cierre IS NULL OR fecha_cierre >= fecha_creacion)
);
```

Listado 1. Definición DDL de la tabla solicitud (extracto de Script_Modelo.sql).

Pruebas de integridad

La integridad se respalda con una batería en Pruebas_Integridad.sql que combina una prueba positiva (inserción válida que debe ser aceptada) con veinte pruebas negativas (operaciones que vulneran intencionalmente alguna restricción y deben ser rechazadas). Cada prueba negativa se envuelve en SAVEPOINT ... ROLLBACK TO SAVEPOINT, lo que permite ejecutar todas en una sola transacción. El detalle se documenta en Evidencias_Validacion.pdf.

Modelamiento No Estructurado

Entre las tres opciones del enunciado (Árbol de Decisión, Clustering K-Means y JSON + JSON Schema), se eligió la Opción C porque se alinea directamente con la naturaleza del caso: el enunciado indica explícitamente que las solicitudes incluyen "datos semiestructurados (JSON con geolocalización, metadatos)", por lo que el





JSON Schema no es un ejercicio académico sino el mecanismo natural para garantizar la calidad de esos datos en producción.

Estructura del JSON metadatos

- **geolocalizacion:** coordenadas WGS84 del incidente más la precisión estimada del GPS en metros.
- **canal_origen:** metadatos del cliente que ingresó la solicitud (IP, user agent, versión de app móvil).
- **atributos:** bloque polimórfico con discriminador 'tipo' que determina qué propiedades adicionales son válidas.

JSON Schema con discriminador

El archivo JSON_Schema.json (Draft 2020-12) define la estructura completa con seis subesquemas, uno por cada tipo de solicitud, activados mediante una cadena de allOf + if/then/else sobre atributos.tipo. Cuando el discriminador vale BACHE, el subesquema exige profundidad_cm, ancho_cm y calzada con sus rangos y dominios; cuando vale LUMINARIA, exige numero_poste y tipo_falla. Cada subesquema usa additionalProperties: false, impidiendo colar campos no declarados.

Las coordenadas se restringen al territorio chileno (latitud entre -56.0 y -17.5; longitud entre -75.7 y -66.4), rechazando automáticamente puntos erróneos del GPS.

Validación ejecutable

El validador validar_json.py ejecuta tres niveles de prueba:

- **Meta-validación:** el propio Schema se valida contra la meta-schema de Draft 2020-12 sin errores.
- **Validación masiva:** los 400 registros sintéticos se validan uno a uno. Resultado: 400/400 aceptados (100%).





- **Pruebas negativas:** doce instancias inválidas, cada una vulnerando una regla distinta. Resultado: 12/12 rechazadas, cubriendo ocho categorías de error.

Validación y Evaluación

Frente de validación	Evaluaciones	Cobertura
Pruebas de integridad relacional	21 (1 positiva + 20 negativas)	PK, FK, UNIQUE, NOT NULL, CHECK
Validación masiva JSON Schema	400 registros sintéticos	Estructura completa del JSONB
Pruebas negativas JSON Schema	12 casos diseñados	8 categorías de error distintas
Verificación de patrones	5 patrones inyectados	Utilidad analítica del dataset
TOTAL	437 evaluaciones	100% reproducibles

Tabla 4. Resumen de validaciones automatizadas ejecutadas sobre el modelo.

Resultados de las consultas analíticas

Se ejecutaron las siete consultas analíticas de Consultas.sql sobre el dataset sintético. La figura 2 muestra la matriz de solicitudes por sector y tipo, que responde directamente a la necesidad "analizar patrones de reclamos por sector y tipo" del enunciado.

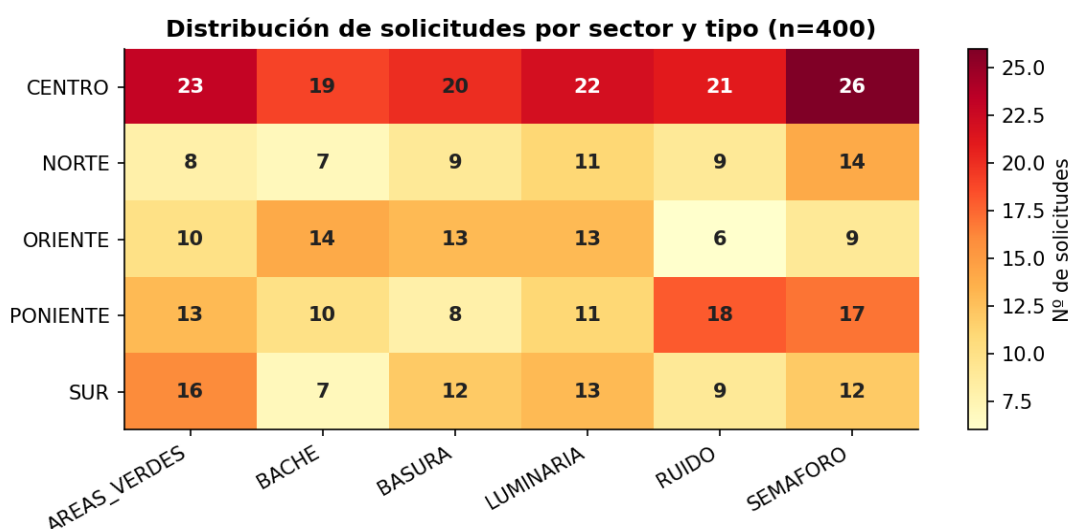


Figura 2. Distribución de solicitudes por sector y tipo. El sector CENTRO concentra el 33,5% del volumen.





El análisis de cumplimiento de SLA por unidad municipal (figura 3) responde a la necesidad "reducir tiempos de respuesta". La línea punteada marca la meta del 70%.

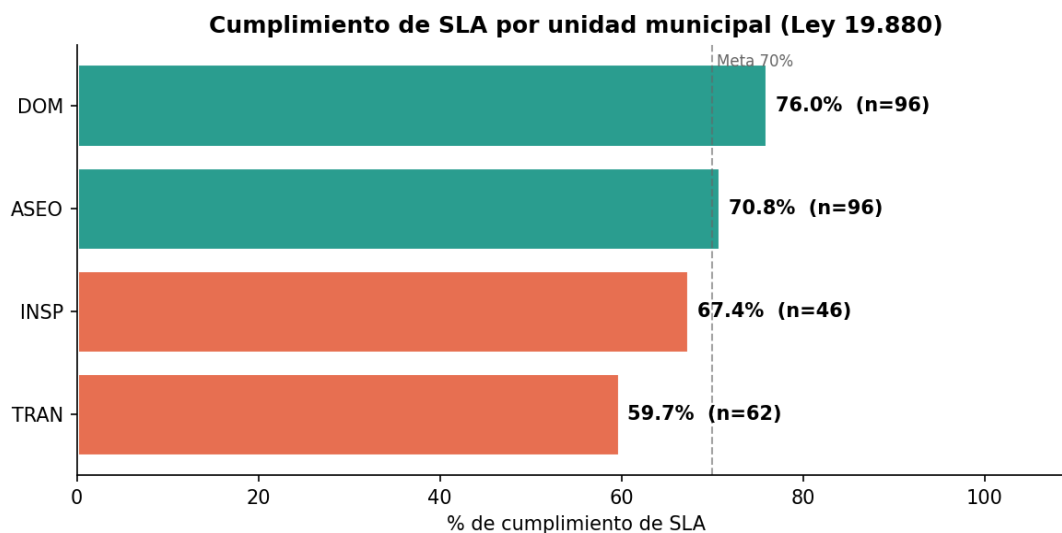


Figura 3. Cumplimiento de SLA por unidad municipal según los plazos de la Ley 19.880.

La figura 4 demuestra empíricamente la utilidad del modelo híbrido: las coordenadas se extraen del campo JSONB metadatos y se grafican espacialmente. Una operación analítica que mezcla columna relacional con campo documental es exactamente el caso de uso que justifica la arquitectura híbrida.



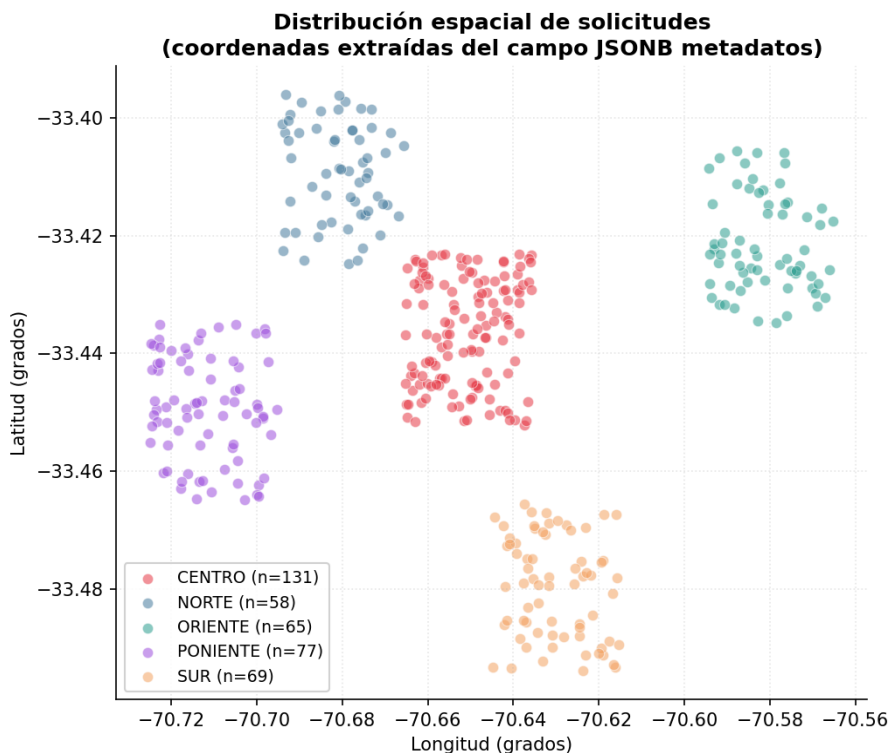


Figura 4. Distribución espacial de las 400 solicitudes, con coordenadas extraídas del JSONB.

Evaluación crítica

Fortalezas

- Integridad demostrada: 33 pruebas negativas distintas + 400 instancias positivas masivas + 5 patrones verificados, todo reproducible.
- Trazabilidad regulatoria completa: historial_estado permite reconstruir cualquier solicitud (Ley 19.880).
- Modelo híbrido genuinamente útil: consultas como "profundidad promedio de baches por sector" son triviales aquí e imposibles o costosísimas en alternativas puramente relacionales.
- Validación bidireccional del JSONB: JSON Schema en la capa de aplicación más CHECK constraints en la base, ofreciendo defensa en profundidad.

Limitaciones





- La denormalización del estado actual exige un trigger de sincronización; sin él, podría divergir en concurrencia.
- Agregar un nuevo tipo de solicitud requiere modificar el JSON Schema y desplegar la nueva versión.
- El URI del adjunto es TEXT sin verificación; un puntero roto a S3 no se detecta a nivel de base.
- Con 150.000 solicitudes anuales, en tres años se acumulan 450.000 filas; se beneficiaría de particionamiento al superar las 500.000.

Oportunidades de mejora

- Implementar el trigger AFTER INSERT ON historial_estado para sincronizar estado_actual_id.
- Índice GIN especializado sobre metadatos->'atributos'->>'tipo' para acelerar análisis por tipo.
- Incorporar PostGIS para clustering geográfico nativo de zonas críticas con polígonos reales.
- Entrenar un clasificador binario de retraso de SLA sobre el dataset, aprovechando los patrones embebidos.
- Particionar solicitud por fecha_creacion cuando el volumen lo justifique.

Impacto en Calidad de Datos

Claridad

Los nombres siguen convenciones snake_case en español sin abreviaturas opacas. Cada tabla y columna crítica lleva COMMENT ON que alimenta el Diccionario de Datos (83 columnas y 23 campos JSONB documentados). Un funcionario nuevo puede entender el modelo sin leer código.

Pertinencia





Todas las variables se justifican por una necesidad explícita del caso. La tabla 1 mapea cada necesidad a las entidades y consultas que la atienden. No hay tablas ni columnas "por si acaso".

Precisión

Se garantiza en tres capas: CHECK constraints en la base (dominios, rangos, regex de RUT y email, coherencia temporal); JSON Schema sobre metadatos (rangos geográficos de Chile, formatos IP, enums por tipo); y validación de patrones sobre el dataset.

Consistencia

Se asegura mediante FK en todas las relaciones, UNIQUE simple (folio, RUT) y compuesto (historial_estado), y ON DELETE CASCADE aplicado únicamente donde es semánticamente correcto: historiales y adjuntos se eliminan con su solicitud; catálogos y ciudadanos nunca se cascadean.

Buenas Prácticas Aplicadas

- **Diccionario de datos formal:** Diccionario_Datos.xlsx con 4 hojas: Resumen (12 tablas), Diccionario maestro (83 columnas), JSONB (23 campos) y Catálogos (24 valores).
- **Control de versiones (Git):** todos los archivos son texto plano versionable. El generador es determinístico (random.seed(42)), lo que permite reproducir exactamente el dataset. Los SQL no contienen datos personales reales.
- **Documentación técnica:** cada archivo incluye encabezado con propósito. El Script_Modelo.sql está organizado en seis secciones comentadas. El Notebook funciona como guía narrativa ejecutable.
- **Validaciones automáticas:** 437 evaluaciones reproducibles sin intervención humana, documentadas en Evidencias_Validacion.pdf.





- **Justificación normativa:** tres leyes chilenas impactan el modelo (21.719, 19.880, 20.285) y cada mecanismo se justifica explícitamente desde ellas.

Conclusiones

El proyecto demuestra que un modelo de datos híbrido sobre PostgreSQL es la respuesta natural a un caso de uso que combina datos transaccionales estructurados con datos semiestructurados de variabilidad legítima. La integridad transaccional y referencial del núcleo relacional, combinada con la flexibilidad controlada del campo JSONB validado por JSON Schema, permite satisfacer simultáneamente las necesidades operativas, analíticas y estratégicas del municipio.

Los 437 puntos de validación automatizada —veinte pruebas negativas SQL, doce casos inválidos del JSON Schema y la verificación empírica de cinco patrones embebidos— constituyen evidencia reproducible de la robustez del diseño y de la utilidad analítica de los datos que el modelo permite gestionar.

La selección de la Opción C (JSON + JSON Schema) para el modelamiento no estructurado se justificó por su alineación directa con la naturaleza del caso: el JSON Schema no es un ejercicio académico agregado sino el mecanismo natural para garantizar la calidad de esos datos en producción.

Las oportunidades de mejora identificadas constituyen una hoja de ruta natural para la evolución del sistema en producción, sin requerir modificaciones estructurales del modelo entregado. El proyecto cumple los siete criterios obligatorios del enunciado y los Resultados de Aprendizaje evaluados, con énfasis en la evidencia ejecutable y reproducible que respalda cada afirmación técnica.

Anexos

Los siguientes archivos forman parte integral de la entrega:

Archivo	Contenido
---------	-----------





Script_Modelo.sql	Esquema DDL completo (12 tablas, índices, catálogos). PostgreSQL 15+.
Modelo_ER.png	Diagrama Entidad-Relación con notación crow's foot.
Datos_Sinteticos.sql	3.076 INSERT: 50 ciudadanos, 400 solicitudes con patrones embebidos.
Pruebas_Integridad.sql	21 pruebas agrupadas por criterio ISO/IEC 25012.
Consultas.sql	4 consultas operativas y 7 analíticas mapeadas a necesidades.
JSON_Schema.json	Schema Draft 2020-12 con discriminador para 6 tipos.
validar_json.py	Validador: 400 positivos + 12 negativos.
Notebook_Modelo.ipynb	Flujo end-to-end con 42 celdas y 3 visualizaciones.
Diccionario_Datos.xlsx	4 hojas: Resumen, Diccionario, JSONB, Catálogos.
Evidencias_Validacion.pdf	Reporte detallado de las 437 evaluaciones.
generar_datos.py	Generador determinístico del dataset (seed=42).

Tabla 5. Archivos técnicos adjuntos a la entrega.

Referencias normativas

- Ley N° 21.719 sobre Protección de Datos Personales. Diario Oficial de la República de Chile, 2024.
- Ley N° 19.880 que Establece Bases de los Procedimientos Administrativos. Diario Oficial, 2003.
- Ley N° 20.285 sobre Acceso a la Información Pública (Transparencia). Diario Oficial, 2008.
- ISO/IEC 25012:2008. Software product Quality Requirements and Evaluation — Data quality model.
- JSON Schema Draft 2020-12. <https://json-schema.org/draft/2020-12/schema>

